日织技术说明文档

Version 1

面向技术学习 简历提炼与 HR 面试表达

项目	当前结论
软件定位	本地优先的 Windows 日程与生活记录桌面应用
实现基线	Electron 44.1.1 与原生 JavaScript
文档日期	2026 年 10 月 20 日
代码规模	约 19,817 行 JavaScript HTML 与 CSS 不含依赖和构建产物

本文只描述当前仓库中能够由代码验证的实现。简历表述应按你的真实贡献范围改成独立负责 主要负责或参与实现。

摘要

日织的技术路线可以概括为 Electron 桌面容器加原生 Web 界面加 Node.js 系统能力，再围绕本地优先原则接入离线语音识别 本地文件存储 局域网媒体上传和桌面发布链路。它不是一个只展示页面的前端练习，而是一个覆盖桌面进程模型 数据迁移 音频处理 媒体处理和软件分发的完整客户端项目。

最值得放入简历的部分有四项。第一是 Electron 主进程 渲染进程和预加载桥接的分层。第二是可迁移的本地数据与媒体体系。第三是 AudioWorklet Worker Threads Qwen3 ASR 和 Silero VAD 组成的离线语音链路。第四是二维码局域网上传 FFmpeg 转码和 NSIS 自动更新形成的桌面产品闭环。

快速技术画像

维度	当前实现	解决的问题
客户端形态	Electron Windows 桌面应用	同时使用 Web 开发效率和桌面系统能力
界面技术	原生 HTML CSS JavaScript 与 DOM 状态驱动	任务 记录 统计 设置和多种弹窗交互
进程边界	Main Preload Renderer Worker	隔离文件系统权限并避免语音推理阻塞界面
数据策略	本地 JSON 按月分片加独立设置文件	隐私优先 可搬迁 可手工备份
离线智能	Qwen3 ASR Int8 加 Silero VAD	本机完成语音转写并按语音段解码
媒体链路	临时 HTTP 服务 二维码上传 FFmpeg 处理	手机在同一局域网向电脑归档照片与视频
表格导入	SheetJS 加 CSV TSV 解析	导入课表并转换为可编辑任务
发布链路	electron-builder NSIS electron-updater	生成 Windows 安装包并从 GitHub Releases 更新
质量基线	Node 语法检查与四组断言测试	覆盖解析 分类 数据迁移和模型迁移

推荐项目定位

适合写成 **本地优先的 Windows 个人效率桌面应用**。如果你确实完成了主要模块，可以进一步描述为覆盖桌面客户端架构 本地数据层 离线 AI 和系统集成的个人项目。

不建议把它包装成云端微服务项目 全栈 SaaS 平台或大模型训练项目，因为当前代码没有服务端业务系统 数据库集群 模型训练或在线多用户能力。准确的项目定位比堆叠热门术语更容易在追问中站得住。

一 项目需求如何转化为技术路线

日织的关键产品约束是个人数据默认留在本机，同时又要具备语音输入 表格导入 手机媒体上传 桌面提醒和自动更新。技术路线因此围绕本地执行和桌面系统集成展开。

产品要求	技术选择	实现结果
个人记录尽量不出设备	本地 JSON 文件与离线 ASR	常规任务 记录和语音识别不依赖业务服务器
需要 Windows 原生体验	Electron BrowserWindow Tray Dialog Shell	主窗口 浮窗 提醒窗 托盘和系统文件夹访问
语音识别不能卡住界面	AudioWorklet 加 IPC 加 Worker Threads	采集 推送 分段 推理解耦
手机照片进入电脑记录	临时 HTTP 服务 随机令牌 二维码	同一局域网内完成一次性上传会话
视频和动态照片可预览	FFmpeg 静态二进制	生成兼容预览 缩略图并修复部分动态照片
历史数据可以迁移	分月数据文件 路径配置 复制校验 回退	数据 设置 媒体和模型可统一更换存放位置
面向普通用户发布	NSIS 安装与 GitHub Releases 更新	可安装 可创建快捷方式 可升级

二 当前系统架构

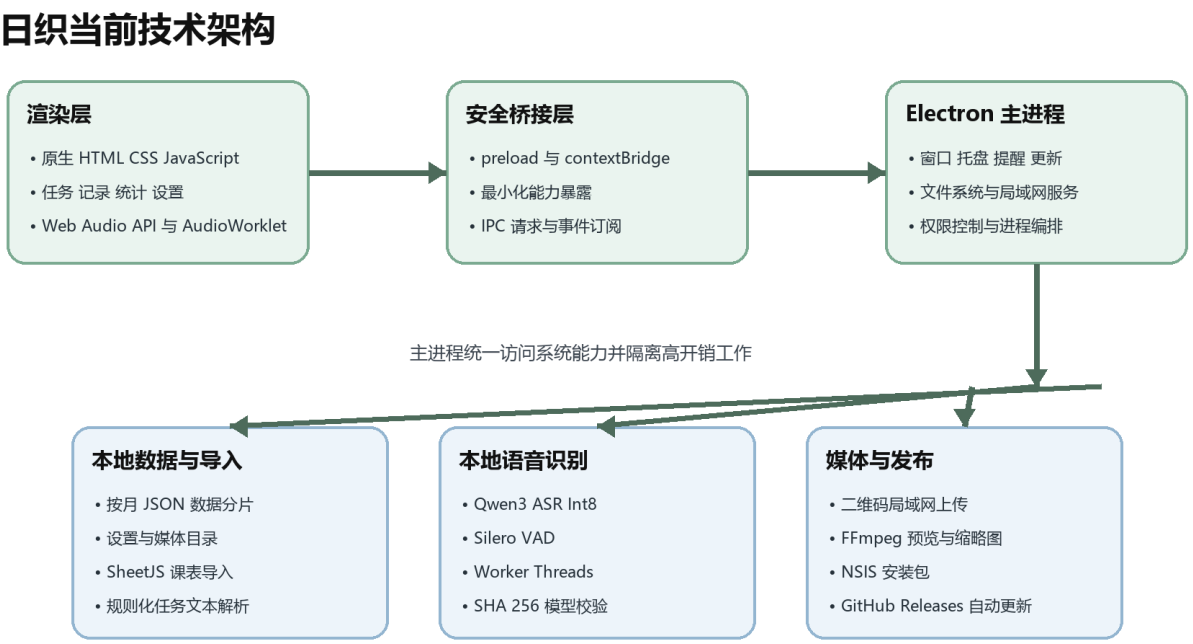


图 1 日织当前技术架构与主要依赖

架构分层

层次	主要文件	职责	边界
渲染层	src renderer.js src index.html src styles.css	界面 状态 交互 统计图和录音入口	不直接使用 Node 文件系统
预加载桥	preload.js floatingTaskPreload.js	通过 contextBridge 暴露受控 API	把 IPC 通道封装成业务方法
主进程	main.js	窗口 托盘 提醒 更新 文件 网络和媒体	持有桌面权限并校验部分消息来源
领域模块	src shared src parsers src storage	日期 时间 分类 解析和持久化	可由 Node 测试直接验证
语音模块	src asr	模型管理 音频分段 Worker 推理	高开销推理离开 UI 线程
构建发布	package.json	NSIS 打包 ASAR 资源配置 自动更新	面向 Windows x64 发布

进程间的数据方向

1. 渲染层把用户操作转换为受控方法调用，例如 saveData parseTaskText startUploadSession 和 asr start。
2. 预加载脚本只暴露明确的方法，不把 ipcRenderer 或 Node API 整体暴露给页面。
3. 主进程负责访问文件系统 启动 HTTP 服务 调用 FFmpeg 管理更新并协调 ASR Worker。
4. 处理结果通过 invoke 返回值或事件通道发送给渲染层，渲染层更新本地状态并重新渲染对应区域。

三 Electron 桌面应用实现

主进程与多窗口

主窗口承载完整应用。浮动任务窗口使用无边框透明窗口 显示当前任务和倒计时，并支持置顶 贴边收起和跨工作区显示。提醒窗口使用临时 HTML 展示队列化通知。系统托盘负责隐藏和恢复主窗口，关闭主窗口时由用户决定退出或继续在托盘运行。

这种设计体现了桌面应用与普通网页的差异。应用生命周期 窗口位置 屏幕工作区 托盘 置顶和后台运行都由 Electron 主进程管理，页面只负责展示和业务交互。

安全桥接与权限控制

措施	当前做法	价值
渲染隔离	主窗口和浮窗启用 contextIsolation 并关闭 nodeIntegration	页面脚本不能直接获得 Node 权限
最小桥接	contextBridge 只暴露 whbr 和 floatingTask 业务 API	缩小攻击面并让调用语义更清晰
来源校验	更新 媒体 ASR 和浮窗等通道检查 sender	避免其他 WebContents 调用敏感能力
麦克风权限	仅允许当前主窗口申请单一音频媒体权限	拒绝摄像头和非主窗口请求
输入约束	颜色 长度 文件扩展名 日期和路径均有校验	降低异常输入进入系统能力层的风险

需要继续加强的地方是 IPC 来源校验尚未覆盖所有数据和文件类通道，参数校验也仍以手写条件为主。后续可统一使用通道白名单 请求结构校验和固定的响应错误结构。

原生前端与状态组织

渲染层没有引入 React Vue 或大型状态管理库，而是使用原生 DOM 事件 全局内存状态和局部 render 函数完成页面。统计图同样由 CSS conic-gradient 和动态 DOM 片段生成。这个方案依赖少 打包简单，但 renderer.js 已达到约 7,821 行，后续维护成本会随功能继续上升。

交互主题	实现要点

交互主题	实现要点
任务时间线	按日期筛选 排序 拖拽 冲突检测 当前任务高亮和跨日拆分
撤销重做	维护删除操作的 undo 和 redo 栈，并与文本编辑器原生撤销区分
记录编辑	维护脏状态，切换页面或日期前询问保存 丢弃或取消
周期统计	按日 周 月 季 年计算日期范围，聚合任务计划时长并绘制比例图
自定义设置	分类 标签 专注周期 快捷键 模板和存储路径都写入 profile

四 任务时间与解析能力

时间模型

任务使用日期 开始时间 结束时间 顺序和分类等字段表示。00:00 到 00:00 被定义为全天任务。结束时间早于开始时间时视为跨日区间，界面保存前会把它拆分成当天到 00:00 和次日从 00:00 开始的两个片段，并用 splitParentId 保留关系。

冲突检测把普通区间和跨日区间转换为可比较的分钟区间，再用半开区间条件判断重叠。因此前一任务 10:00 结束和后一任务 10:00 开始不会被误判为冲突。

自然语言任务解析

任务文本解析不是调用在线大模型，而是使用规则解析。解析器识别今天 明天 后天 下周几 中文数字 上午下午语境 时间范围 任务标题 详情和分类，并输出 missingFields 与 warnings 供界面确认。多任务文本按连续时间段切分，还会根据前一任务结束时间修正缺少时段描述的表达。

示例输入 明天下午三点到五点写论文 算学习任务

解析结果 日期 2026 09 19 时间 15:00 到 17:00 标题 写论文 分类 学习

课表和表格导入

输入	处理	输出
XLSX XLS	SheetJS 读取工作簿 合并单元格补全 表头映射 日期推断 批注提取	任务 生活记录和课表模板候选
CSV TSV	自行处理 BOM 引号 转义和分隔符	与表格统一的数据行结构
课程表布局	识别星期列 时间列和跨行课程文本	七天课程模板供用户确认
普通任务表	识别日期 时间 标题 地点 类型 计划和状态	规范化任务对象

导入后先进入可编辑表格，再由用户确认写入，这是一种重要的产品安全设计。解析器可以做自动化，但最终写入仍保留人工校验。

五 本地数据与迁移设计

文件布局

数据层以文件系统为中心。任务和记录按月份拆分，用户设置与 profile 单独保存，媒体和语音模型使用独立目录。应用启动时读取所有月度文件并按 id 去重，保存时根据任务日期和记录日期范围重新生成对应月份的数据。

```
memory
  rizhi-data-YYYY-MM.json
  rizhi-settings.json
  images
    YYYY-MM
      YYYY-MM-DD
  models
    asr
      model-id
```

统一存放位置迁移

5. 迁移前确认没有语音下载 识别或媒体上传正在进行，并拒绝磁盘根目录 嵌套路径和非空目标目录。
6. 先迁移语音模型，再迁移数据 设置 媒体和其他目录内容。跨磁盘时使用复制后删除代替 rename。
7. 目标端读取 JSON 并校验数据文件，模型迁移还会执行完整性验证。
8. 如果后续步骤失败，代码尝试把模型位置回退到旧目录，并把清理失败保留为警告。

这里体现了比普通文件复制更完整的工程意识，包括路径包含关系判断 符号链接拒绝 目标冲突保护 跨盘兼容 校验 回退和清理警告。

数据方案的取舍

方面	当前优势	后续风险
JSON 可读性	用户可手工查看 备份和迁移	字段演进和复杂查询需要更多代码
按月分片	避免所有历史数据集中在单个文件	每次保存仍会重写活动月份文件
无数据库	无需安装和迁移数据库	数据量大后筛选 聚合和并发写入能力有限
直接写文件	实现简单	普通保存还没有临时文件加原子替换和自动备份
本地优先	隐私边界清晰 离线可用	跨设备同步需要额外的冲突解决方案

六 离线语音识别链路

离线语音是当前项目技术含量最高的链路之一。界面通过 Web Audio API 获取麦克风音频，AudioWorklet 在音频渲染线程把多声道数据混为单声道并重采样到 16 kHz，再按帧传给渲染层。预加载脚本复制 Float32Array 的底层缓冲区，通过 IPC 发送到主进程，主进程再交给 Worker Thread 中的 sherpa-onnx。

识别流水线

阶段	技术	关键处理
音频采集	getUserMedia	选择输入设备 开启回声消除 降噪和自动增益
实时处理	AudioWorklet	单声道混音 线性重采样 帧切分和可转移缓冲区
进程传输	contextBridge 与 IPC	校验采样率和 Float32Array 后传递副本
任务隔离	Node Worker Threads	请求响应超时 状态事件和异常退出处理
语音分段	Silero VAD	检测语音段 限制单段长度并生成局部结果
离线解码	Qwen3 ASR 0.6B Int8 与 sherpa-onnx	CPU 单线程离线识别 最长录音 90 秒
模型管理	HTTPS 下载 SHA 256 校验 分阶段安装	可信主机限制 临时目录 校验后提升和失败清理

为什么要使用 Worker

语音模型加载和推理属于高 CPU 与高内存任务。如果直接在渲染线程执行，输入和动画会卡顿。如果直接堆在主进程，也可能影响窗口和系统事件。当前实现把模型运行时放入 Worker Thread，主进程只负责编排请求跟踪和事件转发，因此界面 主进程和推理任务各自保持清晰职责。

模型安装安全

- 下载只接受 HTTPS 且主机限制为 GitHub 及其发布资源域名。
- 压缩包和 VAD 文件先进入随机临时目录，解压后再校验实际运行文件。
- 关键 ONNX 和 tokenizer 文件同时校验文件大小与 SHA 256。

- 最终目录提升前会保留旧目录备份，失败时恢复，启动时清理遗留下载目录。

  这部分可以在技术面试中作为供应链完整性和大文件安装可靠性的实例，而不应只说集成了语音模型。

七 局域网媒体与桌面发布

二维码局域网上传

用户发起上传时，主进程在 0.0.0.0 的随机端口启动临时 HTTP 服务，从物理网卡候选中选择局域网 IPv4 地址，并生成包含随机 24 字节令牌的上传 URL 和二维码。会话 15 分钟过期，新的会话会关闭旧会话。上传内容先进入 staging 目录，用户确认后才移动到对应记录日期。

控制点	当前实现
会话隔离	crypto.randomUUID 加 base64url 随机令牌
可用性诊断	记录 TCP HTTP 事件 网卡候选和本机回环探测
输入限制	总请求上限约 350 MB 文件扩展名白名单和安全文件名
生命周期	单活动会话 15 分钟超时 主动取消和 staging 清理
用户确认	预览与勾选后才归档到正式媒体目录

该方案适合家庭或个人可信局域网。它使用 HTTP 而不是 HTTPS，访问控制主要依赖难以猜测的临时 URL，因此不应把它描述成公网文件服务。

媒体兼容处理

FFmpeg 用于把视频转为 H.264 AAC MP4 预览，生成图片和视频缩略图，并把 HEIC 等浏览器不易直接显示的格式转换为 JPEG。代码还识别 Android Motion Photo 的 XMP 信息和 MP4 负载，也支持图片加独立视频组成的动态照片。ffmpeg-static 在打包时从 ASAR 中解包，确保安装版能够直接启动可执行文件。

打包与更新

能力	实现
开发运行	electron .
目录打包	electron-builder --dir
Windows 安装包	NSIS x64 可选择安装路径并创建桌面和开始菜单快捷方式

能力	实现
自动更新	electron-updater 检查 GitHub Releases 用户确认后下载并安装
资源处理	应用图标作为额外资源 FFmpeg 二进制使用 asarUnpack

八 工程质量与当前边界

已验证的质量基线

2026 年 10 月 20 日在当前仓库执行 npm run check，主进程 预加载脚本 渲染脚本 ASR 模块 共享工具 解析器 存储模块和测试文件的语法检查通过。任务文本解析 分类规范化 本地数据迁移和 ASR 模型迁移四组测试也全部通过。

测试范围	覆盖内容	当前状态
任务文本与时间	相邻区间 跨日 全天 中文日期与时间 多任务解析	通过
任务分类	默认分类 自定义分类 重命名 颜色与专注周期	通过
本地数据迁移	分月文件 媒体路径 标签兼容和无关文件保留	通过
ASR 模型迁移	完整性校验 路径持久化 冲突保护和失败回退	通过

需要如实说明的不足

现状	影响	Version 2 建议
renderer.js 和 main.js 体积较大	功能之间的修改容易互相影响	按页面 领域服务 IPC 和窗口管理继续拆分
测试主要是 Node 断言脚本	缺少真实窗口和用户流程回归	增加 Playwright Electron 端到端测试与媒体集成测试
普通数据保存不是原子写入	进程异常中断可能留下半写文件	临时文件写入 fsync 后 rename 并保留滚动备份
IPC 校验不完全统一	个别通道缺少一致的 sender 和参数验证	集中注册通道并引入 schema 校验
局域网上传使用 HTTP	只适合可信局域网和短期会话	明确网络边界 或增加设备确认与本地 TLS
没有代码签名和 CI 配置证据	安装信誉和自动发布能力仍有限	加入 Windows 代码签名 持续集成和发布门禁
没有性能基准	不能声称启动速度或识别延迟达到某指标	建立启动 内存 识别耗时和大数据量基准

九 简历写法

技术栈关键词

Electron Node.js JavaScript HTML5 CSS3 Context Bridge IPC Worker Threads Web Audio API AudioWorklet sherpa-onnx Qwen3 ASR Silero VAD FFmpeg SheetJS QR Code HTTP File System Crypto electron-builder NSIS electron-updater GitHub Releases

项目简介推荐版本

日织是一款本地优先的 Windows 日程与生活记录桌面应用，使用 Electron 构建，支持任务规划 周期统计 课表导入 离线语音转写 局域网媒体归档 浮动任务提醒 本地数据迁移和自动更新。

可选简历要点

- 基于 Electron 设计主进程 预加载桥和渲染层分工，通过 contextIsolation contextBridge 与 IPC 封装文件系统 窗口 托盘 更新和媒体能力。
- 设计本地优先的数据层，将任务与记录按月拆分为 JSON 文件，并实现数据 设置 媒体和离线模型的统一路径迁移 校验 冲突保护与失败回退。
- 集成 Qwen3 ASR Int8 和 Silero VAD，使用 AudioWorklet 完成音频采集与 16 kHz 重采样，使用 Worker Threads 隔离离线推理，并对约 840 MB 模型执行可信源下载与 SHA 256 完整性校验。
- 实现二维码局域网上传，使用随机令牌和短时会话接收手机媒体，通过 FFmpeg 生成图片 视频和动态照片的预览与缩略图。
- 实现 XLSX XLS CSV TSV 课表导入和中文任务文本规则解析，支持日期 时段 标题 详情 分类 跨日拆分与时间冲突检测。
- 使用 electron-builder 生成 Windows NSIS 安装包，并通过 electron-updater 接入 GitHub Releases 更新流程。

建议根据岗位只选三到五条。客户端岗位优先写 Electron 数据与系统集成，AI 应用岗位优先写离线语音，前端岗位优先写复杂交互 时间算法和可视化。

不能直接写的表述

不建议	原因	可信替代
搭建高并发后端系统	当前没有后端业务服务和并发指标	实现局域网临时 HTTP 上传服务

不建议	原因	可信替代
训练或微调大模型	当前是集成现成离线模型	集成并工程化部署 Qwen3 ASR Int8
实现毫秒级实时识别	没有延迟基准且模型按 VAD 语音段解码	实现离线分段语音转写和局部结果反馈
采用数据库分库分表	当前是按月 JSON 文件分片	设计可迁移的本地分月存储
完善的自动化测试体系	当前只覆盖四类核心逻辑	为解析和迁移等高风险模块编写断言测试
完全安全	仍有 IPC 与局域网边界可加强	采用上下文隔离 最小桥接和模型完整性校验

十 向 HR 讲清楚项目

三十秒版本

日织是我做的一款 Windows 本地日程和生活记录软件。它不仅有任务和统计界面，还把语音识别 照片视频归档 数据迁移 桌面浮窗和自动更新做成了完整产品。技术上用 Electron，数据默认保存在用户电脑，语音也在本机识别，所以我在这个项目里练到的是桌面客户端架构 本地数据可靠性和离线 AI 工程化。

一分钟版本

这个项目解决的是个人日程和生活记录分散的问题。我选择 Electron，是因为它既能快速做复杂界面，也能访问 Windows 的窗口 托盘 文件系统和安装更新能力。软件把任务和记录按月保存在本地，用户可以统一迁移数据 照片视频和语音模型。语音部分不是调用云端接口，而是用 AudioWorklet 采集音频，再交给独立 Worker 中的离线模型识别，避免卡住界面。手机照片则通过同一局域网扫码上传，电脑端用 FFmpeg 生成兼容预览。这个项目让我完整经历了从界面 进程通信 数据设计 到打包发布和测试的桌面软件开发过程。

三分钟展开顺序

9. 先说用户问题和本地优先原则，不要一上来报模型名。
10. 再说 Electron 的四层结构，即界面 预加载桥 主进程和 Worker。
11. 选择一个最难的模块展开，推荐语音链路或统一迁移。
12. 说明你如何处理失败情况，例如模型校验失败回退 路径冲突保护 上传过期和解析后人工确认。
13. 最后主动说当前不足和下一步计划，显示你能区分可运行与可长期维护。

HR 常见问题

问题	回答重点
你在项目里具体做了什么	按真实贡献说模块和决策，不要笼统说负责全部开发
最难的地方是什么	语音链路涉及音频线程 IPC Worker VAD 模型安装和错误恢复
为什么不用云端语音	个人记录隐私 离线可用和长期成本是主要原因，代价是模型体积与本机算力
为什么选择 Electron	适合复杂 Web 界面并能快速获得 Windows 系统能力，代价是包体和内存

问题	回答重点
如何保证数据不丢	分月文件 迁移前校验 目标冲突保护 模型完整性校验和回退；普通保存仍计划增加原子写与备份
项目如何验证	核心模块有语法检查和解析 分类 数据迁移 模型迁移测试，UI 端到端测试是下一步

十一 技术面试追问准备

为什么不直接让页面访问 Node

Electron 页面可能包含大量 DOM 和用户输入。如果打开 nodeIntegration，页面脚本一旦出现注入问题就能直接操作文件和进程。日织关闭 nodeIntegration，启用 contextIsolation，只通过 preload 暴露明确业务方法，从而把高权限能力集中在主进程。

为什么当前使用 JSON 而不是 SQLite

当前产品是单用户 本地写入 数据量相对可控，JSON 便于备份 查看和迁移，按月分片也限制了单文件规模。代价是复杂查询 原子性和大数据量性能较弱。如果记录规模继续增长，或者需要全文检索和跨设备同步，SQLite 会更合适。

为什么语音模型还需要 VAD

Qwen3 ASR 在这里是离线识别器，并不天然等于持续流式识别。Silero VAD 先判断哪些音频属于语音，再把完整语音段送去解码。这样可以限制单次推理长度 减少无声输入并逐段返回结果，同时保留最大录音时长作为资源保护。

如何处理大模型文件下载失败

模型不直接写入最终目录。下载 解压 VAD 补充和哈希校验都在随机 staging 目录完成，只有完整验证后才提升为正式目录。正式目录存在时先备份，提升失败再恢复。这样能避免半个模型被误判为已安装。

如何解释局域网上传的安全边界

上传页面只在用户主动打开后短期存在，URL 带高熵随机令牌并在 15 分钟后失效，文件经过大小 类型和文件名校验，内容先进入暂存目录。但传输仍是局域网 HTTP，因此设计目标是可信家庭或个人网络，不是公网服务。

如果继续做 Version 2

优先级	改进	预期收益

优先级	改进	预期收益
高	把 renderer.js 按任务 记录 统计 设置和语音控制器拆分	降低回归范围并便于单元测试
高	为数据保存加入原子替换 版本化迁移和滚动备份	提高异常退出时的数据可靠性
高	统一 IPC sender 检查 参数 schema 和错误码	形成可审计的桌面权限边界
中	使用 Playwright 驱动 Electron 关键用户流程	覆盖窗口 文件选择 录音状态和上传确认
中	评估 SQLite 与全文检索	支撑更多年份数据和组合筛选
中	增加性能基准与本地日志	定位启动 推理 媒体转码和保存耗时
中	加入 Windows 代码签名和 CI 发布	降低安装警告并提高发布可重复性

附录一 代码模块索引

文件或目录	职责	面试可讲内容
package.json	依赖 脚本 NSIS 打包和 GitHub 发布配置	软件分发与资源解包
main.js	窗口 托盘 更新 媒体服务器 FFmpeg 和 IPC	桌面主进程编排
preload.js	受控业务 API 与事件桥	Context Bridge 和最小权限
src renderer.js	界面状态 交互 统计 任务与记录	复杂原生前端与时间算法
src storage localDataStore.js	分月存储 兼容迁移和统一目录迁移	本地数据可靠性
src parsers	课表文件和中文任务文本解析	规则解析与人工确认
src shared	日期 时间 分类和标签领域规则	共享纯函数和兼容层
src asr audioCapture.js	麦克风设备与 AudioWorklet 音频图	Web Audio API
src asr audioWorkletProcessor.js	混音 重采样和帧输出	实时音频处理
src asr asrService.js	Worker 生命周期 请求跟踪和事件	线程隔离与超时
src asr asrWorker.js	VAD 分段和 sherpa-onnx 解码	离线语音推理
src asr modelManager.js	模型下载 解压 校验 安装和清理	大文件供应链完整性
tests	解析 分类 数据和模型迁移测试	高风险逻辑回归

附录二 面试前自检

- □ 我能画出 Renderer Preload Main Worker 四层关系，并解释每层为什么存在。
- □ 我能从麦克风开始，完整讲到 AudioWorklet IPC VAD Worker 和最终文本。
- □ 我能解释 JSON 按月分片的优点，也能坦诚原子写和查询能力的不足。
- □ 我能解释随机令牌 15 分钟会话 暂存目录和人工确认如何保护媒体上传。
- □ 我能讲清跨日任务为什么要拆分，以及相邻任务为什么不算冲突。
- □ 我能说明哪些功能由现有测试覆盖，哪些还需要端到端测试。
- □ 简历里的每个动词都与我的真实贡献一致，并且没有虚构性能指标。

结论

日织当前最有辨识度的能力不是单个页面，而是把本地优先桌面架构 离线语音 文件迁移 局域网媒体和 Windows 发布连接成了一个可运行产品。简历和面试应围绕工程问题 技术选择 失败处理和实际边界来表达。这样既能体现技术深度，也不会在追问时陷入无法证明的指标或名词。